

2.1.4 答案与解析

单项选择题

01. C

线性表是由具有相同数据类型的有限数据元素组成的，数据元素是由数据项组成的。

02. B

线性表定义的要求为：相同数据类型、有限序列。选项 C 的元素个数是无穷个，错误；选项 A 集合中的元素没有前后驱关系，错误；选项 D 属于一种存储结构，本题要求选出的是一个具体的线性表，不要将二者混为一谈。只有选项 B 符合线性表定义的要求。

03. A

线性表中，除最后一个（或第一个）元素外，每个元素都只有一个后继（或前驱）元素。

04. A

线性表中的第一个元素没有直接前驱，最后一个元素没有直接后继；当线性表中仅有一个元素时，该元素既没有直接前驱，又没有直接后继。

2.2.4 答案与解析

一、单项选择题

01. A

顺序表不像链表那样要在结点中存放指针域，因此存储密度较大，选项 A 正确。选项 B 和 C 是链表的优点。选项 D 是错误的，比如对于树形结构，顺序表显然不如链表表示起来方便。

02. C

顺序表是顺序存储的线性表，表中所有元素的类型必须相同，且必须连续存放。一维数组中的元素可以不连续存放；此外，栈、队列和树等逻辑结构也可利用一维数组表示，但它与顺序表不属于相同的逻辑结构。在顺序表中，逻辑上相邻的元素物理位置上也相邻。

03. A

本题易误选 B。注意，存取方式是指读/写方式。顺序表是一种支持随机存取的存储结构，根据起始地址加上元素的序号，可以很方便地访问任意一个元素，这就是随机存取的概念。

04. C

随机存取是指在 $O(1)$ 的时间访问下标为 i 的元素，所需时间与顺序表中的元素个数 n 无关。

05. B

顺序表所占的存储空间 = 表长 $\times$ sizeof(元素的类型)，表长和元素的类型显然会影响存储空间的大小。若元素为结构体类型，则元素中各字段的类型也会影响存储空间的大小。

06. D

题干实际要求能最快存取第 $i-1$ 、 i 和 $i+1$ 个元素值。选项 A、B、C 都只能从头结点依次顺序查找，时间复杂度为 $O(n)$ ；只有顺序表可以按序号随机存取，时间复杂度为 $O(1)$ 。

07. A

只有顺序表可以按序号随机存取，且在最后进行插入和删除操作时不需要移动任何元素。

08. C

对于 I，解析略；对于 II，在最后位置插入新结点不需要移动元素，时间复杂度为 $O(1)$ ；对于 III，被删结点后的结点需要依次前移，时间复杂度为 $O(n)$ ；对于 IV，需要后移 $n-i$ 个结点，时间复杂度为 $O(n)$ 。

09. C

对于 II，顺序表只需要 3 次交换操作；链表则需要分别找到两个结点前驱，第 4 个结点断链后再插入到第 2 个结点后，效率较低。对于 III，需依次顺序访问每个元素，时间复杂度相同。

10. C

需要将元素 $a_{i+1} \sim a_n$ 依次前移一位，共移动 $n-(i+1)+1=n-i$ 个元素。

11. C

在第 i 个位置插入一个元素，需要移动 $n-i+1$ 个元素，时间复杂度为 $O(n)$ 。

12. C

对 n 个元素进行排序的时间复杂度最小也要 $O(n)$ （初始有序时），通常为 $O(n \log_2 n)$ 或 $O(n^2)$ ，通过第 8 章学习后会更理解。B 和 D 显然错误。顺序表支持按序号的随机存取方式。

13. B

线性表元素的序号是从 1 开始，而在第 $n+1$ 个位置插入相当于在表尾追加。

14. D

顺序存储需要连续的存储空间，在申请时需申请 $n+m$ 个连续的存储空间，然后将线性表原来的 n 个元素复制到新申请的 $n+m$ 个连续的存储空间的前 n 个单元。

15. D

对于顺序存储的有序表，查找指定值元素可以采用顺序查找法或折半查找法，平均时间复杂度最少为 $O(\log n)$ 。插入指定值元素需要先找到插入位置，然后将该位置及之后的元素依次后移一个位置，最后将指定值元素插入到该位置，平均时间复杂度为 $O(n)$ 。删除第 i 个元素需要将该元素之后的全部元素依次前移一个位置，平均时间复杂度为 $O(n)$ 。获取第 i 个元素只需直接根据下

标读取对应的数组元素即可，时间复杂度为 $O(1)$ 。

2.3.8 答案与解析

一、单项选择题

01. B

两种存储结构适用于不同的场合，不能简单地谁好谁坏，I 错误。链式存储用指针表示逻辑结构，而指针的设置是任意的，因此比顺序存储结构能更方便地表示各种逻辑结构，II 正确。在顺序存储中，插入和删除结点需要移动大量元素，效率较低，III 的描述刚好相反。顺序存储结构既能随机存取又能顺序存取，而链式结构只能顺序存取，IV 正确。

02. B

首先直接排除 A 和 D。散列存储通过散列函数映射到物理空间，不能反映数据之间的逻辑关系，排除 C。链式存储能方便地表示各种逻辑关系，且插入和删除操作的时间复杂度为 $O(1)$ 。

03. A

链式存储设计时，各个不同结点的存储空间可以不连续，但结点内的存储单元地址必须连续。

04. D

顺序存储方式同样适用于存储图和树，I 错误。删除表尾结点时，必须从头开始找到表尾结点的前驱，其时间与表长有关，II 错误。循环单链表中最后一个结点的指针不是 NULL，而是指向头结点，整个链表形成一个环，因此不存在空指针，III 正确。有序单链表只能依次查找插入位置，时间复杂度为 $O(n)$ ，IV 正确。队列需要在表头删除元素，表尾插入元素，采用带尾指针的循

环链表较为方便, 插入和删除的时间复杂度都为 $O(1)$, V 正确。

05. A

对于 A, 在单链表和顺序表上实现的时间复杂度都为 $O(n)$, 但后者要移动很多元素, 因此在单链表上实现效率更高。对于 B 和 D, 顺序表的效率更高。C 无区别。

06. C

s 插入后, q 成为 s 的前驱, 而 p 成为 s 的后继, 选 C。

注 意

可能有读者认为选项 C 中的两条语句交换后才正确。实际上, 因为本题插入位置的前后结点都有指针指示 (这与前面介绍的插入操作是不同的), 所以选项 C 中的语句顺序并不会造成断链。在此提醒读者在学习过程中一定要多动脑思考, 而不要生搬硬套。

07. D

若先建立链表, 然后依次插入建立有序表, 则每插入一个元素就需遍历链表寻找插入位置, 即直接插入排序, 时间复杂度为 $O(n^2)$ 。若先将数组排好序, 然后建立链表, 建立链表的时间复杂度为 $O(n)$, 数组排序的最好时间复杂度为 $O(n\log_2 n)$, 总时间复杂度为 $O(n\log_2 n)$ 。故选 D。

08. C

先遍历长度为 m 的单链表, 找到该单链表的尾结点, 然后将其 next 域指向另一个单链表的首结点, 其时间复杂度为 $O(m)$ 。

09. C

单链表设置头结点的目的是方便运算的实现, 主要好处体现在: 第一, 有头结点后, 插入和删除数据元素的算法就统一了, 不再需要判断是否在第一个元素之前插入或删除第一个元素; 第二, 不论链表是否为空, 其头指针是指向头结点的非空指针, 链表的头指针不变, 因此空表和非空表的处理也就统一了。

10. B

删除单链表的最后一个结点需置其前驱结点的指针域为 NULL, 需要从头开始依次遍历找到该前驱结点, 需要 $O(n)$ 的时间, 与表长有关。其他操作均与表长无关, 读者可自行模拟。

11. B, A

在带头结点的单链表中, 头指针 head 指向头结点, 头结点的 next 域指向第一个元素结点, head->next==NULL 表示该单链表为空。在不带头结点的单链表中, head 直接指向第一个元素结点, head==NULL 表示该单链表为空。

12. D

线性表有顺序存储和链式存储两种存储结构。若采用链式存储结构, 则删除元素 a_{50} 不需要移动元素; 若采用顺序存储结构, 则需要依次移动 50 个元素。

13. B

当采用头插法建立单链表时, 数组后面的元素插入到单链表 L 的最前端, 所以 L 中的元素次序与数组 a 的元素次序相反。

14. C

A 显然错误。B 表中第一个元素和最后一个元素不满足题设要求。双链表能很方便地访问前驱和后继, 故删除和插入数据较为方便, C 正确。D 未考虑顺序存储的情况。

15. D

为了在 p 之前插入结点 q, 可以将 p 的前一个结点的 next 域指向 q, 将 q 的 next 域指向

p, 将 q 的 prior 域指向 p 的前一个结点, 将 p 的 prior 域指向 q。仅 D 满足条件。

16. A

与上一题的分析基本类似, 只不过这里是删除一个结点, 注意将 p 的前、后两结点链接起来。关键是要保证在结点指针的修改过程中不断链!

注意, 请读者仔细对比上述两题, 弄清双链表的插入和删除方法。

17. A

结点 A 和 B 分别由指针 p 和 q 指示, 但结点 C 仅能由 p->next 间接指示, 因此在改变 p->next 和 p->next->prior 之前, 必须先将 q->next 指向结点 C, 即①要在③和④前面, 且④要在③前面 (因为若先执行③, 则④相当于 q->prior 指向其自身, 显然矛盾)。故只能选 A。

18. C

当在双链表的两个结点 (分别用第一个、第二个结点表示) 之间插入一个新结点时, 需要修改四个指针域, 分别是: 新结点的前驱指针域, 指向第一个结点; 新结点的后继指针域, 指向第二个结点; 第一个结点的后继指针域, 指向新结点; 第二个结点的前驱指针域, 指向新结点。

19. B

设单链表递增有序, 首先要在单链表中找到第一个大于 x 的结点的直接前驱 p, 在 p 之后插入该结点。查找的时间复杂度为 $O(n)$, 插入的时间复杂度为 $O(1)$, 总时间复杂度为 $O(n)$ 。

20. D

在插入和删除操作上, 单链表和双链表都不用移动元素, 都很方便, 但双链表修改指针的操作更为复杂, A 错误。双链表中可以快速访问任何一个结点的前驱和后继结点, D 正确。

21. C

带头结点的循环单链表 L 为空表时, 满足 $L \rightarrow \text{next} = L$, 即头结点的指针域与 L 的值相等, 而不是头结点的指针域与 L 的地址相等。注意, 带头结点的单循环链表中不存在空指针。

22. D

循环双链表 L 判空的条件是头结点 (头指针) 的 prior 和 next 域都指向它自身。

23. A

在链表的末尾插入和删除一个结点时, 需要修改其相邻结点的指针域。而寻找尾结点及尾结点的前驱结点时, 只有带头结点的双循环链表所需要的时间最少。

24. C

对于 A, 删除尾结点 *p 时, 需要找到 *p 的前一个结点, 时间复杂度为 $O(n)$ 。对于 B, 删除首结点 *p 时, 需要找到 *p 结点, 这里没有直接给出头结点指针, 而通过尾结点的 prior 指针找到 *p 结点的时间复杂度为 $O(n)$ 。对于 D, 删除尾结点 *p 时, 需要找到 *p 的前一个结点, 时间复杂度为 $O(n)$ 。对于 C, 执行这四种算法的时间复杂度均为 $O(1)$ 。

25. B

要求用 $O(1)$ 的时间将两个循环单链表头尾相接, 并未指明哪个链表接在另一个链表之后, 所以对两个链表都要在 $O(1)$ 的时间找到头结点和尾结点。因此, 两个指针应都指向尾结点。

26. A

在循环单链表中, 删除首元结点后, 要保持链表的循环性, 因此需要找到首元结点的前驱。当链表带头结点时, 其前驱就是头结点, 因此不论是表头指针还是表尾指针, 删除首元结点的时间都为 $O(1)$ 。当链表不带头结点时, 其前驱是尾结点, 因此, 若有表尾指针, 就可在 $O(1)$ 的时间找到尾结点; 若只有表头指针, 则需要遍历整个链表找到尾结点, 时间为 $O(n)$ 。

27. D

对于一个空循环单链表, 有 $\text{head} \rightarrow \text{next} == \text{head}$, 推理 $\text{head} \rightarrow \text{next} \rightarrow \text{next} == \text{head} \rightarrow \text{next} == \text{head}$ 。对含有一个元素的循环单链表, 头结点 (头指针 head 指示) 的 next 域指向这个唯一的元素结点, 该元素结点的 next 域指向头结点, 因此也有 $\text{head} \rightarrow \text{next} \rightarrow \text{next} = \text{head}$ 。

28. D

对于两种双链表, 删除首结点的时间复杂度都是 $O(1)$ 。对于非循环双链表, 删除尾结点的时间复杂度是 $O(n)$; 对于循环双链表, 删除尾结点的时间复杂度是 $O(1)$ 。

29. D

对于 A, 在最后一个元素之后插入元素的情况与普通单链表相同, 时间复杂度为 $O(n)$; 而删除第一个元素时, 为保持单循环链表的性质 (尾结点指向第一个结点), 要先遍历整个链表找到尾结点, 再做删除操作, 时间复杂度为 $O(n)$ 。对于 B, 双链表的情况与单链表的相同, 一个是 $O(n)$, 一个是 $O(1)$ 。对于 C, 在最后一个元素之后插入一个元素, 要遍历整个链表才能找到插入位置, 时间复杂度为 $O(n)$; 删除第一个元素的时间复杂度为 $O(1)$ 。对于 D, 与 A 的分析对比, 有尾结点的指针, 省去了遍历链表的过程, 因此时间复杂度均为 $O(1)$ 。

30. B

静态链表采用数组表示, 因此需要预先分配较大的连续空间, 静态链表同时还具有一般链表的特点, 即插入和删除不需要移动元素。

31. B

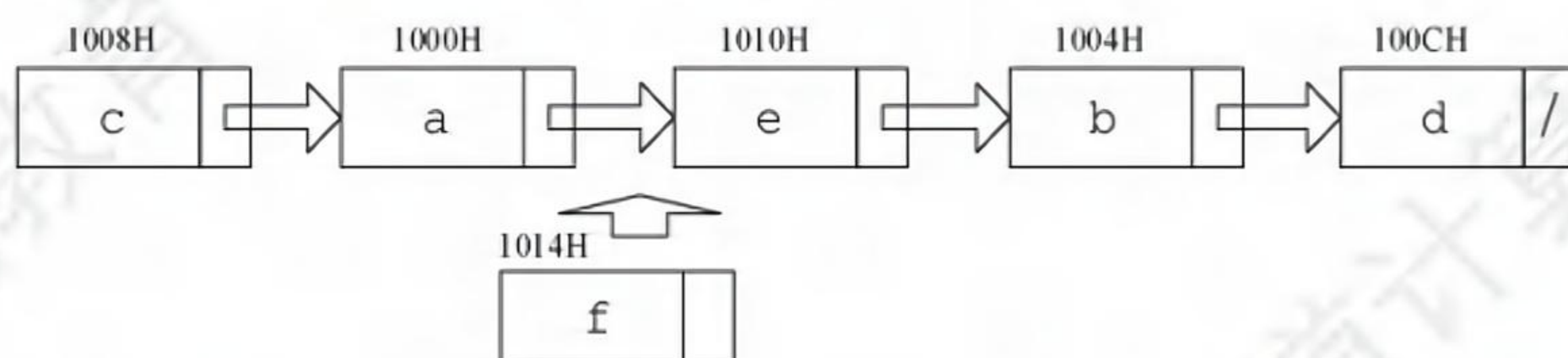
静态链表的存储空间虽然是顺序分配的, 但元素的存储不是顺序的, 查找时仍然需要按链依次进行, 而插入、删除都不需要移动元素。静态链表的存储空间是一次性申请的, 能容纳的最大元素个数在定义时就已确定。由于并非每个空间都存储了元素, 因此会造成存储空间的浪费。

32. D

A 的第二句代码, 相当于将 p 前驱结点的后继指针指向其自身, 错误; B 和 C 的第一句代码, 相当于将 p 后继结点的前驱指针指向其自身, 错误。只有 D 正确。

33. D

根据存储状态, 单链表的结构如下图所示。



其中“链接地址”是指结点 next 所指的内存地址。当结点 f 插入后, a 指向 f , f 指向 e , e 指向 b 。显然 a 、 e 和 f 的“链接地址”分别是 f 、 b 和 e 的内存地址, 即 1014H 、 1004H 和 1010H 。

34. D

如图 1 所示, 要删除带头结点的非空单循环链表中的第一个元素, 就要先用临时指针 q 指向待删结点, $q = h \rightarrow \text{next}$; 然后将 q 从链表中断开, $h \rightarrow \text{next} = q \rightarrow \text{next}$ (这一步也可写成 $h \rightarrow \text{next} = h \rightarrow \text{next} \rightarrow \text{next}$); 此时要考虑一种特殊情况, 若待删结点是链表的尾结点, 即循环单链表中只有一个元素 (p 和 q 指向同一个结点), 如图 2 所示, 则在删除后要将尾指针指向头结点, 即 $\text{if } (p == q) p = h$; 最后释放 q 结点即可, 答案选 D。

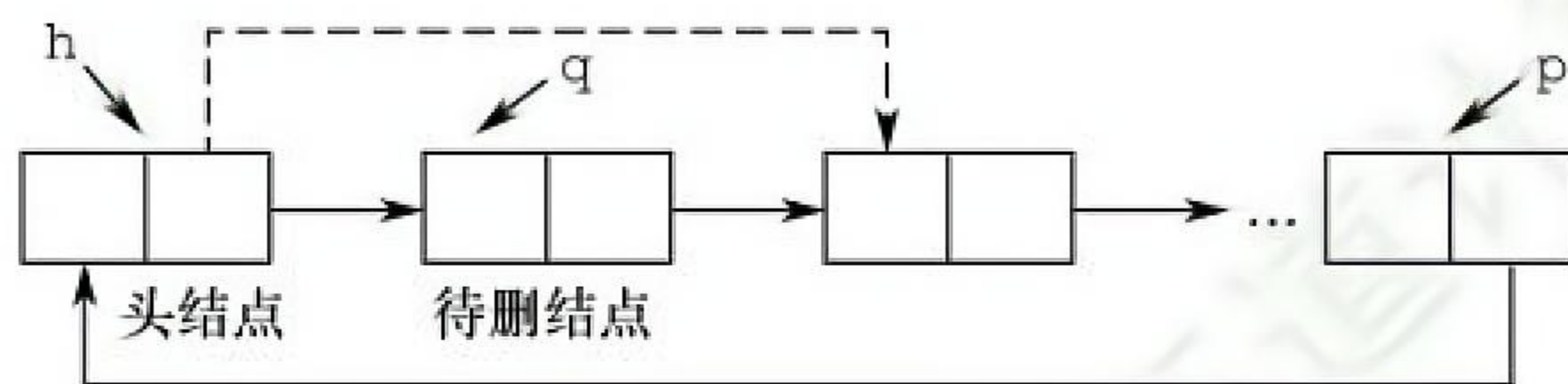


图 1

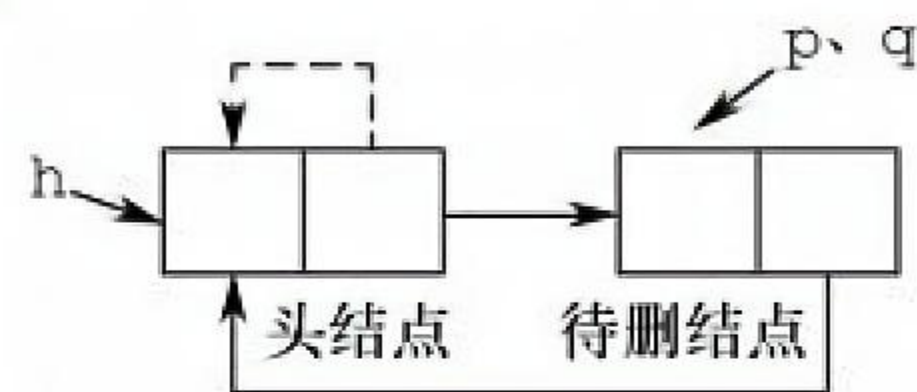
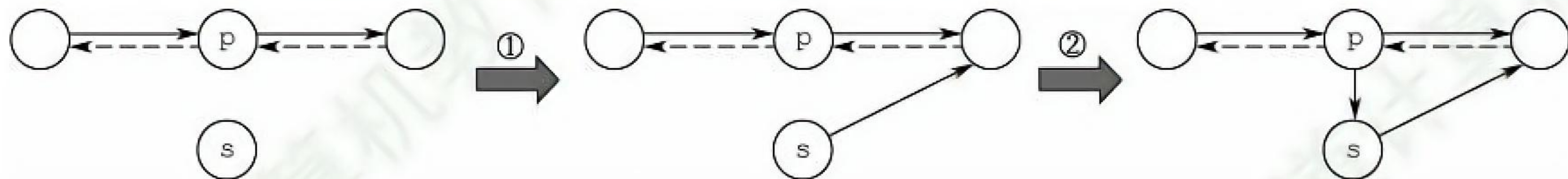


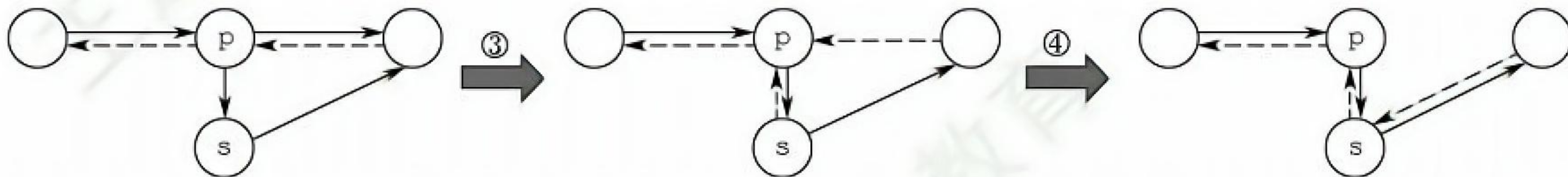
图 2

35. C

链表的插入操作要保证不会造成断链, 画图再依次判断选项。执行完语句 “① $s \rightarrow next = p \rightarrow next$;
② $p \rightarrow next = s$;” 后的结构如下图所示 (虚线表示 prev, 实线表示 next)。



对于 A, $s \rightarrow next \rightarrow prev = p$, 错误。对于 B, $p \rightarrow next \rightarrow prev = s$, 让 s 的 prev 指向 s , 错误。对于 D, 两句代码均错误。对于 C, 执行完语句 “③ $s \rightarrow prev = s \rightarrow next \rightarrow prev$;
④ $s \rightarrow next \rightarrow prev = s$;” 后的结构如下图所示, 满足插入的要求。



3.1.5 答案与解析

一、单项选择题

01. B

栈和队列的逻辑结构都是相同的，都属于线性结构，只是它们对数据的运算不同。

02. C

首先栈是一种线性表，所以 B、D 错。按存储结构的不同可分为顺序栈和链栈，但不可以把栈局限在某种存储结构上，所以 A 错。栈和队列都是限制存取点的线性结构。

03. B

基本操作是指该结构最核心、最基本的运算，其他较复杂的操作可通过基本操作实现。删除栈底元素不属于栈的基本运算，但它可以通过调用栈的基本运算求得。

04. C

数组下标范围为 $0 \sim n-1$ ，初始时 top 为 1，第一个元素入栈后， top 为 0，即 top 指向栈顶元素。栈向高地址方向增长，所以入栈时应先将指针 top 加 1，然后存入元素 x ，C 正确。

05. B

数组下标范围为 $1 \sim n$ ，初始时 top 为 1，表示 top 指向栈顶元素的下一个元素。栈向高地址方向增长，所以入栈时应先存入元素 x ，然后将指针 top 加 1，B 正确。

06. A

数组下标范围为 $1 \sim n$ ，初始时 top 为 $n+1$ ，表示 top 指向栈顶元素。栈向低地址方向增长，所以入栈时应先将指针 top 减 1，然后存入元素 x ，A 正确。

07. A

每个元素需要 1 个存储单元，所以每入栈一次 top 加 1，出栈一次 top 减 1。指针 top 的值依次为 1001H, 1002H, 1001H, 1002H, 1001H, 1002H, 1001H, 1002H。

08. A

顺序栈采用数组存储，数组的大小是固定的，不能动态地分配大小。和顺序栈相比，链栈的最大优势在于它可以动态地分配存储空间。

09. C

对于双向循环链表，不管是表头指针还是表尾指针，都可以很方便地找到表头结点，方便在表头做插入或删除操作。而单循环链表通过尾指针可以很方便地找到表头结点，但通过头指针找尾结点需要遍历一次链表。对于 C，插入和删除结点后，找尾结点所需的时间为 $O(n)$ 。

10. C

链栈采用不带头结点的单链表表示时，进栈操作在首部插入一个结点 x (即 $x \rightarrow \text{next} = \text{top}$)，插入完后需将 top 指向该插入的结点 x 。请思考当链栈存在头结点时的情况。

11. D

这里假设栈顶指针指向的是栈顶元素，所以选 D；而 A 中首先将 top 指针赋给了 x ，错误；B 中没有修改 top 指针的值；C 为 top 指针指向栈顶元素的上一个元素时的答案。